

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №5
по курсу «Параллельная обработка данных»

Сортировка чисел на GPU. Свертка, сканирование, гистограмма.

Выполнил: Орусский В.Р.
Группа: М8О-406Б-21
Преподаватель: А.Ю. Морозов

Условие

Цель работы: ознакомление с фундаментальными алгоритмами GPU: свертка (reduce), сканирование (blelloch scan) и гистограмма (histogram). Реализация одной из сортировок на CUDA. Использование разделяемой и других видов памяти. Исследование производительности программы с помощью утилиты nvprof (ncu).

Вариант 1. Битоническая сортировка:

Входные данные. В первых четырех байтах записывается целое число n - длина массива чисел `int`, далее следуют n чисел (тип зависит от варианта задания).

Программное и аппаратное обеспечение

GPU:

- Название NVIDIA GeForce GTX 1660 TI
- Compute capability: 7.5
- Графическая память: 2147155968
- Разделяемая память: 49152
- Константная память: 65536
- Количество регистров на блок: 32
- Максимальное количество нитей: (1024, 1024, 64)
- Максимальное количество блоков: (2147483647, 65535, 65535)
- Количество мультипроцессоров: 24

Сведения о системе:

- Процессор: Intel(R) Core(TM) i5-10600KF CPU @ 4.5000MHz
- ОЗУ: 16 ГБ
- SSD M2 1 ТБ

Программное обеспечение:

- OS: Windows 10 + WSL (Ubuntu 20.04)
- IDE: Visual Studio Code
- Компилятор: nvcc

Метод решения

Битоническая сортировка — это рекурсивный алгоритм, который сортирует массив, преобразуя его в битоническую последовательность, то есть такую, которая сначала монотонно возрастает, а затем монотонно убывает. Сначала массив делится на две половины, каждая из которых рекурсивно сортируется в противоположных направлениях: первая половина по возрастанию, вторая — по убыванию. Затем эти половины объединяются через процесс битонического слияния, где пары элементов сравниваются и обмениваются местами так, чтобы вся последовательность стала отсортированной по возрастанию. Слияние повторяется на уменьшающихся подпоследовательностях, пока весь массив не будет полностью упорядочен. Алгоритм эффективен для параллельных вычислений, так как операции сравнения и обмена можно выполнять одновременно.

Сложность сортировки с учетом распараллеливания - $O(\log(n) * \log(n))$.

Описание программы

1. В_step: Выполняет шаг битонического слияния: сравнивает и обменивает элементы в массиве на основе их позиций, расстояния между элементами и размера текущего рассматриваемого подмассива.
2. kernel_shared: Выполняет битоническую сортировку с использованием общей памяти GPU.
3. kernel_global: Выполняет битоническое слияние непосредственно в глобальной памяти GPU.

Результаты

Размер сетки ядра \ Длина массива	100к элементов, мс	1кк элементов, мс	10кк элементов, мс
CPU	241,81	856,20	17247,70
<<<(64, 512)>>>	22,45	233,34	4290,43
<<<(128, 512)>>>	21,98	228,22	4274,75
<<< (256, 512)>>>	20,02	219,43	4268,88
<<< (512, 512)>>>	19,84	218,18	4245,73

Профилировщик

```
kernel_global(int *, int, int, int) (512, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.9
Section: GPU Speed Of Light Throughput
-----
Metric Name      Metric Unit Metric Value
-----
DRAM Frequency   Ghz          11.02
SM Frequency      Ghz          2.25
Elapsed Cycles    cycle        6492
Memory Throughput %           5.11
DRAM Throughput   %           5.11
Duration          us           2.88
L1/TEX Cache Throughput %       16.92
L2 Cache Throughput %           1.68
SM Active Cycles  cycle       1605.39
Compute (SM) Throughput %        5.86
-----

Section: Launch Statistics
-----
Metric Name      Metric Unit Metric Value
-----
Block Size              512
Function Cache Configuration CachePreferNone
Grid Size              512
Registers Per Thread    register/thread 20
Shared Memory Configuration Size Kbyte          8.19
Driver Shared Memory Per Block Kbyte/block     1.02
Dynamic Shared Memory Per Block byte/block          0
Static Shared Memory Per Block byte/block          0
# SMs                  SM              80
Stack Size             1024
Threads                thread         262144
# TPCs                 40
Enabled TPC IDs        all
Uses Green Context     0
Waves Per SM           2.13
-----

Section: Occupancy
-----
Metric Name      Metric Unit Metric Value
-----
Block Limit SM          block          24
Block Limit Registers    block           5
Block Limit Shared Mem    block           8
Block Limit Warps        block           3
Theoretical Active Warps per SM warp           48
Theoretical Occupancy    %            100
Achieved Occupancy       %            83.05
Achieved Active Warps Per SM warp           39.86
-----

Section: GPU and Memory Workload Distribution
-----
Metric Name      Metric Unit Metric Value
-----
Average DRAM Active Cycles cycle          1622
Total DRAM Elapsed Cycles cycle        253952
Average L1 Active Cycles cycle        1605.39
Total L1 Elapsed Cycles cycle        538652
Average L2 Active Cycles cycle        1186.97
Total L2 Elapsed Cycles cycle        176256
Average SM Active Cycles cycle        1605.39
Total SM Elapsed Cycles cycle        538652
Average SMSP Active Cycles cycle        1531.13
Total SMSP Elapsed Cycles cycle       2154608
-----
```

```
kernel shared(int *, int, int, int) (512, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.9
Section: GPU Speed Of Light Throughput
```

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	11.21
SM Frequency	Ghz	2.29
Elapsed Cycles	cycle	63031
Memory Throughput	%	1.02
DRAM Throughput	%	0.37
Duration	us	27.52
L1/TEX Cache Throughput	%	12.00
L2 Cache Throughput	%	0.25
SM Active Cycles	cycle	5302.62
Compute (SM) Throughput	%	4.27

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		512
Function Cache Configuration		CachePreferNone
Grid Size		512
Registers Per Thread	register/thread	25
Shared Memory Configuration Size	Kbyte	65.54
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	Kbyte/block	8.19
# SMs	SM	80
Stack Size		1024
Threads	thread	262144
# TPCs		40
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		2.13

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	24
Block Limit Registers	block	4
Block Limit Shared Mem	block	7
Block Limit Warps	block	3
Theoretical Active Warps per SM	warp	48
Theoretical Occupancy	%	100
Achieved Occupancy	%	81.68
Achieved Active Warps Per SM	warp	39.20

Section: GPU and Memory Workload Distribution

Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	1132
Total DRAM Elapsed Cycles	cycle	2468864
Average L1 Active Cycles	cycle	5302.62
Total L1 Elapsed Cycles	cycle	4986032
Average L2 Active Cycles	cycle	2458.09
Total L2 Elapsed Cycles	cycle	1712672
Average SM Active Cycles	cycle	5302.62
Total SM Elapsed Cycles	cycle	4986032
Average SMSP Active Cycles	cycle	5247.32
Total SMSP Elapsed Cycles	cycle	19944128

Выводы

В данной лабораторной работе я реализовал параллельную битоническую сортировку на CUDA, создав два варианта: с использованием глобальной и разделяемой памяти. Реализация на глобальной памяти оказалась простой, тогда как работа с разделяемой памятью потребовала усилий из-за ее ограниченного размера, что вынудило разработать более сложные алгоритмы. Тем не менее, применение разделяемой памяти обеспечило прирост производительности. При удачном избежании конфликтов банков памяти время выполнения программы можно сократить еще больше.